

Trial Division

Concept: Trial division is the most straightforward method for determining if a number n is prime (checking for primality). The basic idea is to try dividing n by every integer from 2 up to $\sqrt{n}$.

- If n is divisible by any of these numbers, it is **composite** (not prime).
- If you reach $\sqrt{n}$ without finding a divisor, n is **prime**.

Why $\sqrt{n}$? If n has a factor larger than $\sqrt{n}$, it must also have a matching factor smaller than $\sqrt{n}$. Therefore, checking up to the square root is sufficient.

Example: Is 29 prime?

- $\sqrt{29} \approx 5.38$. We only need to check integers 2, 3, 4, and 5.
- $29 \pmod{2} \neq 0$
- $29 \pmod{3} \neq 0$
- $29 \pmod{4} \neq 0$
- $29 \pmod{5} \neq 0$
- **Result:** 29 is Prime.

Java Code:

```
public class TrialDivision {
    public static boolean isPrime(int n) {
        // 0 and 1 are not prime numbers
        if (n <= 1) return false;

        // Check from 2 to square root of n
        for (int i = 2; i * i <= n; i++) {
            if (n % i == 0) {
                return false; // Found a divisor, not prime
            }
        }
        return true; // No divisors found, it is prime
    }

    public static void main(String[] args) {
        System.out.println("Is 29 prime? " + isPrime(29)); // true
        System.out.println("Is 15 prime? " + isPrime(15)); // false
    }
}
```

The Sieve of Eratosthenes

Concept: The Sieve is a highly efficient algorithm to find **all** prime numbers up to a specific limit N . Instead of checking each number individually (like trial division), it iteratively marks the multiples of each prime as composite.

Steps:

1. Create a list of consecutive integers from 2 through N .
2. Start with the smallest prime, $p = 2$.
3. Mark all multiples of p (i.e., $2p, 3p, 4p...$) as composite (not prime).
4. Find the next number in the list that is not marked. Let this be the new p .
5. Repeat until $p^2 > N$.

Example: Primes up to 10

1. List: 2, 3, 4, 5, 6, 7, 8, 9, 10
2. Start at **2**. Cross out multiples: 4, 6, 8, 10.
 - *Remaining:* 2, 3, 5, 7, 9
3. Next is **3**. Cross out multiples: 6 (already done), 9.
 - *Remaining:* 2, 3, 5, 7
4. Next is **5**. $5^2 = 25$, which is > 10 , so we stop.
5. **Primes:** 2, 3, 5, 7.

Java Code:

```
import java.util.Arrays;

public class SieveOfEratosthenes {
    public static void sieve(int n) {
        // boolean array to mark primes. true = prime, false = composite
        boolean[] isPrime = new boolean[n + 1];
        Arrays.fill(isPrime, true); // Assume all are prime initially
        isPrime[0] = false;
        isPrime[1] = false;

        for (int p = 2; p * p <= n; p++) {
            // If isPrime[p] is not changed, then it is a prime
            if (isPrime[p]) {
                // Update all multiples of p
                for (int i = p * p; i <= n; i += p) {
                    isPrime[i] = false;
                }
            }
        }

        // Print all prime numbers
        System.out.print("Primes up to " + n + ": ");
        for (int i = 2; i <= n; i++) {
            if (isPrime[i]) {
                System.out.print(i + " ");
            }
        }
        System.out.println();
    }

    public static void main(String[] args) {
        sieve(30);
    }
}
```

Prime Factorization

Concept: Prime factorization is the process of breaking a composite number down into the product of prime numbers. According to the Fundamental Theorem of Arithmetic, every integer greater than 1 is either a prime itself or can be represented as the product of prime numbers in a unique way.

Example: Factorize 60

1. Check 2: $60 \div 2 = 30$. (Factors: 2)

2. Check 2 again: $30 \div 2 = 15$. (Factors: 2, 2)
3. Check 2: 15 not divisible.
4. Check 3: $15 \div 3 = 5$. (Factors: 2, 2, 3)
5. Check 5: $5 \div 5 = 1$. (Factors: 2, 2, 3, 5)
6. **Result:** $60 = 2^2 \times 3 \times 5$

Java Code:

```
import java.util.ArrayList;
import java.util.List;

public class PrimeFactorization {

    public static List<Integer> getPrimeFactors(int n) {
        List<Integer> factors = new ArrayList<>();

        // Loop from 2 to sqrt(n)
        for (int i = 2; i * i <= n; i++) {
            // While n is divisible by i, add i and divide n
            while (n % i == 0) {
                factors.add(i);
                n /= i;
            }
        }

        // If n is still greater than 1, the remainder is a prime
        if (n > 1) {
            factors.add(n);
        }

        return factors;
    }

    public static void main(String[] args) {
        int number = 315;
        System.out.println("Prime factors of " + number + ": " + getPrimeFactors(number));
        // Output: [3, 3, 5, 7]
    }
}
```

Counting Divisors

Concept: To find the total number of divisors of n , you could check every number from 1 to n . However, a faster way uses **Prime Factorization**.

If the prime factorization of a number n is:

$$n = p_1^{a_1} \times p_2^{a_2} \times \cdots \times p_k^{a_k}$$

The total count of divisors is:

$$\text{Count} = (a_1 + 1)(a_2 + 1) \dots (a_k + 1)$$

Example: Count divisors of 12

1. Prime Factorization: $12 = 2^2 \times 3^1$

- Exponents are $a_1 = 2$ and $a_2 = 1$.
- Formula: $(2 + 1)(1 + 1) = 3 \times 2 = 6$.
- Verification:** The divisors of 12 are 1, 2, 3, 4, 6, 12. There are exactly 6.

Java Code:

```
public class CountDivisors {
    public static int countDivisors(int n) {
        int totalDivisors = 1;

        // Loop from 2 to sqrt(n)
        for (int i = 2; i * i <= n; i++) {
            if (n % i == 0) {
                int count = 0;
                // Count the power of the current prime factor
                while (n % i == 0) {
                    count++;
                    n /= i;
                }
                // Apply formula: total = (p1 + 1) * (p2 + 1) ...
                totalDivisors *= (count + 1);
            }
        }

        // If n > 1, the remainder is a prime number with power 1
        // So we multiply by (1 + 1) = 2
        if (n > 1) {
            totalDivisors *= 2;
        }

        return totalDivisors;
    }

    public static void main(String[] args) {
        int num = 12;
        System.out.println("Total divisors of " + num + ": " + countDivisors(num));
        // Output for 12: 6 (Divisors are 1, 2, 3, 4, 6, 12)
    }
}
```

GCD (Greatest Common Divisor)

Concept: The GCD of two integers is the largest number that divides both of them without leaving a remainder. The most efficient way to calculate it is the **Euclidean Algorithm**.

Logic: $GCD(a, b) = GCD(b, a \pmod{b})$ until b becomes 0.

[Image of Euclidean algorithm diagram]

Example: GCD(48, 18)

- $48 \pmod{18} = 12$
- $18 \pmod{12} = 6$
- $12 \pmod{6} = 0$
- Stop. The GCD is **6**.

Java Code:

```
public class GCD {  
    // Recursive implementation  
    public static int gcd(int a, int b) {  
        if (b == 0) return a;  
        return gcd(b, a % b);  
    }  
  
    public static void main(String[] args) {  
        System.out.println("GCD of 48 and 18: " + gcd(48, 18));  
    }  
}
```

LCM (Least Common Multiple)

Concept: The LCM is the smallest positive integer that is divisible by both numbers. It is directly related to the GCD by the formula:

$$LCM(a, b) = \frac{|a \times b|}{GCD(a, b)}$$

Example: LCM(4, 6)

1. $GCD(4, 6) = 2$.
2. Product = $4 \times 6 = 24$.
3. $LCM = 24/2 = 12$.

Java Code:

```
public class LCM {  
    public static int gcd(int a, int b) {  
        if (b == 0) return a;  
        return gcd(b, a % b);  
    }  
  
    public static int lcm(int a, int b) {  
        if (a == 0 || b == 0) return 0;  
        return (a * b) / gcd(a, b);  
        // Note: For large numbers, divide before multiplying to avoid overflow:  
        // return (a / gcd(a, b)) * b;  
    }  
  
    public static void main(String[] args) {  
        System.out.println("LCM of 4 and 6: " + lcm(4, 6));  
    }  
}
```

Modular Inverse

Part 1: Modular Inverse using Fermat's Little Theorem

Best for: When the modulus m is a **Prime Number** (e.g., $10^9 + 7$).

Concept: According to Fermat's Little Theorem, if m is a prime number, then for any integer a :

$$a^{m-2} \equiv a^{-1} \pmod{m}$$

This means finding the inverse is as simple as calculating a raised to the power of $m - 2$.

Steps:

1. Verify m is prime.
2. Calculate `power(a, m - 2) % m`.

Java Code (Fermat's Method):

```
public class FermatInverse {

    // Function to find modular inverse of a under modulo m
    public static int modInverse(int a, int m) {
        return modPow(a, m - 2, m);
    }

    // Standard Binary Exponentiation function (O(log m))
    public static int modPow(int base, int exp, int mod) {
        int res = 1;
        base = base % mod;
        while (exp > 0) {
            if ((exp & 1) == 1) res = (res * base) % mod; // If bit is set
            base = (base * base) % mod; // Square the base
            exp >>= 1; // Shift right
        }
        return res;
    }

    public static void main(String[] args) {
        int a = 3, m = 7;
        // 3^(7-2) = 3^5 = 243. 243 % 7 = 5.
        System.out.println("Inverse of " + a + " mod " + m + " is: " + modInverse(a, m));
    }
}
```

Part 2: Modular Inverse using Extended Euclidean Algorithm

Best for: When the modulus m is **NOT Prime** (general case), but a and m are coprime ($GCD(a, m) = 1$).

Concept: This method finds integers x and y such that:

$$ax + my = GCD(a, m)$$

Since we want the inverse, we assume $GCD(a, m) = 1$. The equation becomes:

$$ax + my = 1$$

Taking modulo m on both sides removes my , leaving $ax \equiv 1 \pmod{m}$. Thus, x is the modular inverse.

Java Code (Extended Euclidean):

```
public class EuclideanInverse {
```

```

public static int modInverse(int a, int m) {
    int m0 = m; // Keep original m for final step
    int y = 0, x = 1;

    if (m == 1) return 0; // Edge case

    while (a > 1) {
        // standard Euclidean Algorithm steps
        int q = a / m; // Quotient
        int t = m;

        // Update a and m
        m = a % m;
        a = t;
        t = y;

        // Update x and y
        y = x - q * y;
        x = t;
    }

    // If x is negative, make it positive
    if (x < 0) x += m0;

    return x;
}

public static void main(String[] args) {
    int a = 3, m = 11;
    // Works even though m=11 is prime, but intended for general cases
    System.out.println("Inverse of " + a + " mod " + m + " is: " + modInverse(a, m));
}
}

```

Modular Exponentiation

Concept: Computing huge powers like $x^n \pmod{m}$ efficiently. Instead of calculating x^n (which overflows), we use the "Square and Multiply" method (Binary Exponentiation). This reduces complexity from $O(n)$ to $O(\log n)$.

Example: $3^5 \pmod{7}$

- Binary of exponent 5 is 101 .
- Start result = 1.
- Bit 1: Multiply result by base. Square base.
- Bit 0: Only square base.
- Bit 1: Multiply result by base. Square base.

Java Code:

```

public class ModExponentiation {
    public static long power(long base, long exp, long mod) {
        long res = 1;
        base = base % mod;

        while (exp > 0) {

```

```

        // If exp is odd, multiply base with result
        if ((exp % 2) == 1)
            res = (res * base) % mod;

        // exp must be even now
        exp = exp >> 1; // divide by 2
        base = (base * base) % mod;
    }
    return res;
}

public static void main(String[] args) {
    System.out.println("3^5 mod 7 = " + power(3, 5, 7));
}
}

```

You have covered a very strong core of Number Theory. However, strictly for **LeetCode and Technical Interviews**, there are **three** major concepts missing from your list that appear frequently.

Here are the missing concepts, formatted exactly like your notes.

6. Euclidean Algorithm (GCD & LCM)

Concept: The Greatest Common Divisor (GCD) is the largest number that divides two integers without a remainder. The Least Common Multiple (LCM) is the smallest number that is a multiple of both. This is arguably **the most common** math concept in coding interviews (used in array rotations, string patterns, and flow problems).

Formula:

- **GCD:** $\text{gcd}(a, b) = \text{gcd}(b, a \% b)$ (Recursively reduce until b is 0).
- **LCM:** $(a * b) / \text{gcd}(a, b)$ (Be careful of overflow: compute $(a / \text{gcd}) * b$).

Example: GCD(48, 18)

1. $\text{gcd}(48, 18) \rightarrow \text{gcd}(18, 48 \% 18) \rightarrow \text{gcd}(18, 12)$
2. $\text{gcd}(18, 12) \rightarrow \text{gcd}(12, 18 \% 12) \rightarrow \text{gcd}(12, 6)$
3. $\text{gcd}(12, 6) \rightarrow \text{gcd}(6, 12 \% 6) \rightarrow \text{gcd}(6, 0)$
4. Result: 6

Java Code:

```

public class GCD_LCM {
    // Time Complexity: O(log(min(a, b)))
    public static int gcd(int a, int b) {
        if (b == 0) return a;
        return gcd(b, a % b);
    }

    public static int lcm(int a, int b) {
        if (a == 0 || b == 0) return 0;
        // Divide first to avoid integer overflow
        return (a / gcd(a, b)) * b;
    }

    public static void main(String[] args) {
        System.out.println("GCD of 48, 18: " + gcd(48, 18)); // 6
    }
}

```

```

        System.out.println("LCM of 4, 6: " + lcm(4, 6));    // 12
    }
}

```

Matrix Exponentiation (Fast Fibonacci / Linear Recurrences)

Concept: You already know "Modular Exponentiation" to compute x^n . Matrix Exponentiation uses the same logic to compute the n -th term of a linear recurrence (like Fibonacci) in $O(\log n)$ time instead of $O(n)$.

If you represent the Fibonacci transition as a matrix:

$$\begin{bmatrix} F_{n+1} & F_n \end{bmatrix} = \begin{bmatrix} 1 & 1 & 0 \end{bmatrix} \begin{bmatrix} F_n & F_{n-1} \end{bmatrix}$$

To find the n -th term, you raise the matrix to power $n - 1$:

$$\begin{bmatrix} 1 & 1 & 0 \end{bmatrix}^{n-1}$$

Why use it? If LeetCode asks for the "billionth Fibonacci number" (Climbing Stairs with $n = 10^9$), standard DP ($O(n)$) will Time Limit Exceeded (TLE). This method works in milliseconds.

Java Code:

```

public class MatrixExponentiation {
    static long MOD = 1000000007;

    // Main function to get Nth Fibonacci
    public static long getNthFibonacci(int n) {
        if (n <= 1) return n;

        long[][] base = {{1, 1}, {1, 0}};
        // Raise matrix to power n-1
        long[][] result = matrixPower(base, n - 1);

        // The result is in the top-left cell (F_n) derived from F_1 and F_0
        return result[0][0];
    }

    // Standard Binary Exponentiation for Matrices
    public static long[][] matrixPower(long[][] base, int exp) {
        long[][] res = {{1, 0}, {0, 1}}; // Identity Matrix

        while (exp > 0) {
            if ((exp & 1) == 1) res = multiply(res, base);
            base = multiply(base, base);
            exp >>= 1;
        }
        return res;
    }

    // Multiply two 2x2 matrices
    public static long[][] multiply(long[][] A, long[][] B) {
        long[][] C = new long[2][2];
        for (int i = 0; i < 2; i++) {
            for (int j = 0; j < 2; j++) {
                for (int k = 0; k < 2; k++) {
                    C[i][j] = (C[i][j] + A[i][k] * B[k][j]) % MOD;
                }
            }
        }
    }
}

```

```

        }
    }
}

return C;
}

public static void main(String[] args) {
    // Calculates 100th Fibonacci number instantly
    System.out.println("Fib(100): " + getNthFibonacci(100));
}
}

```

Reservoir Sampling (Randomization)

Concept: How do you select k elements from a stream of data of **unknown size** n such that every element has an equal probability of being chosen? You cannot store all elements (memory constraint).

Algorithm (Algorithm R):

1. Fill the "reservoir" (array) with the first k elements.
2. For the i -th element (where $i > k$):
 - Generate a random index j between 0 and i .
 - If $j < k$, replace `reservoir[j]` with the new element.

Why use it? Classic interview question: "Pick a random line from a huge file that doesn't fit in RAM." (LeetCode 382, 398).

Java Code:

```

import java.util.Random;

public class ReservoirSampling {
    // Select k random items from a stream
    public static int[] selectKItems(int[] stream, int k) {
        int[] reservoir = new int[k];

        // Step 1: Fill reservoir with first k elements
        for (int i = 0; i < k; i++) {
            reservoir[i] = stream[i];
        }

        Random rand = new Random();

        // Step 2: Process the rest of the stream
        for (int i = k; i < stream.length; i++) {
            // Pick a random index from 0 to i
            int j = rand.nextInt(i + 1);

            // If the random index falls inside the reservoir, replace it
            if (j < k) {
                reservoir[j] = stream[i];
            }
        }
        return reservoir;
    }
}

```

```
public static void main(String[] args) {  
    int[] stream = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};  
    int[] sample = selectKItems(stream, 3);  
  
    System.out.print("Random Sample: ");  
    for (int num : sample) System.out.print(num + " ");  
}
```

Modular Arithmetic Formula

- **Modular Addition:** $(a + b) \% m = ((a \% m) + (b \% m)) \% m$
- **Modular Subtraction:** $(a - b) \% m = ((a \% m) - (b \% m) + m) \% m$
- **Modular Multiplication:** $(a * b) \% m = ((a \% m) * (b \% m)) \% m$
- **Modular Division:** $(a / b) \% m = ((a \% m) * (b^{-1} \% m)) \% m$
- **Modular Exponentiation:** $(a ^ b) \% m = ((a \% m) ^ b) \% m$